

Búsqueda Semántica Multimodal con CLIP

Embeddings · Retrieval · Frontier AI

¿Cómo busca una máquina?

El problema de la búsqueda semántica

Búsqueda por palabras clave

- Busca coincidencias exactas de texto
- "perro" ≠ "dog" ≠ "canino"
- No entiende imágenes
- Frágil ante sinónimos



Búsqueda semántica

- Entiende el significado, no las palabras
- "dog" ≈ "perro" ≈ imagen de perro
- Funciona entre texto e imagen
- Robusta a paráfrasis y variaciones

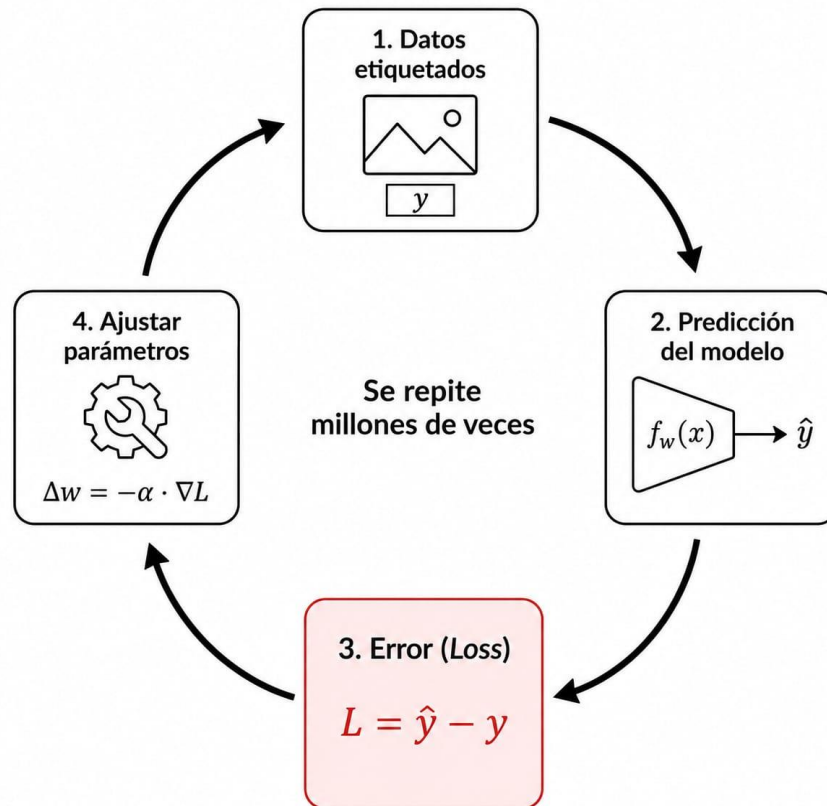
Fundamentos

Cómo aprende una red neuronal

Aprender de ejemplos

El ciclo de entrenamiento

- El modelo no se programa — aprende patrones de los datos
- Necesita ejemplos etiquetados: (input, respuesta correcta)
- Cada ejemplo ajusta levemente los parámetros internos
- Después de millones de ejemplos, el modelo generaliza



Gradiente y optimización

Descender la función de pérdida

Función de pérdida (loss)

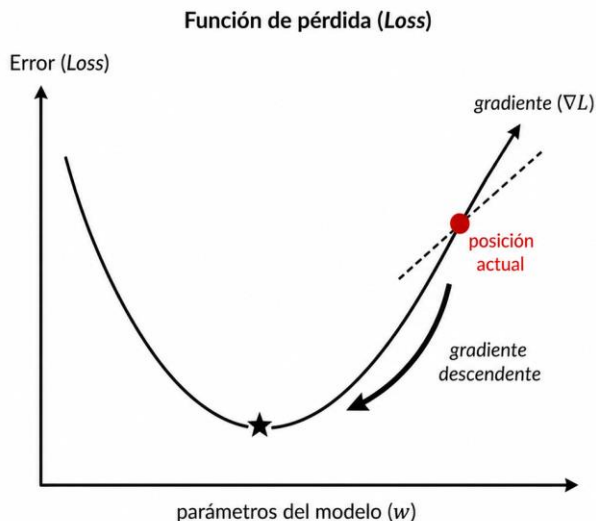
Mide qué tan equivocado está el modelo.
El objetivo es minimizarla.

Gradiente

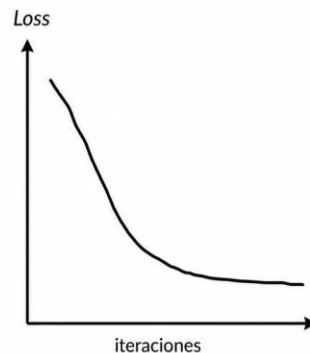
Dirección de mayor crecimiento del error.
Moverse en sentido contrario reduce el error.

Backpropagation

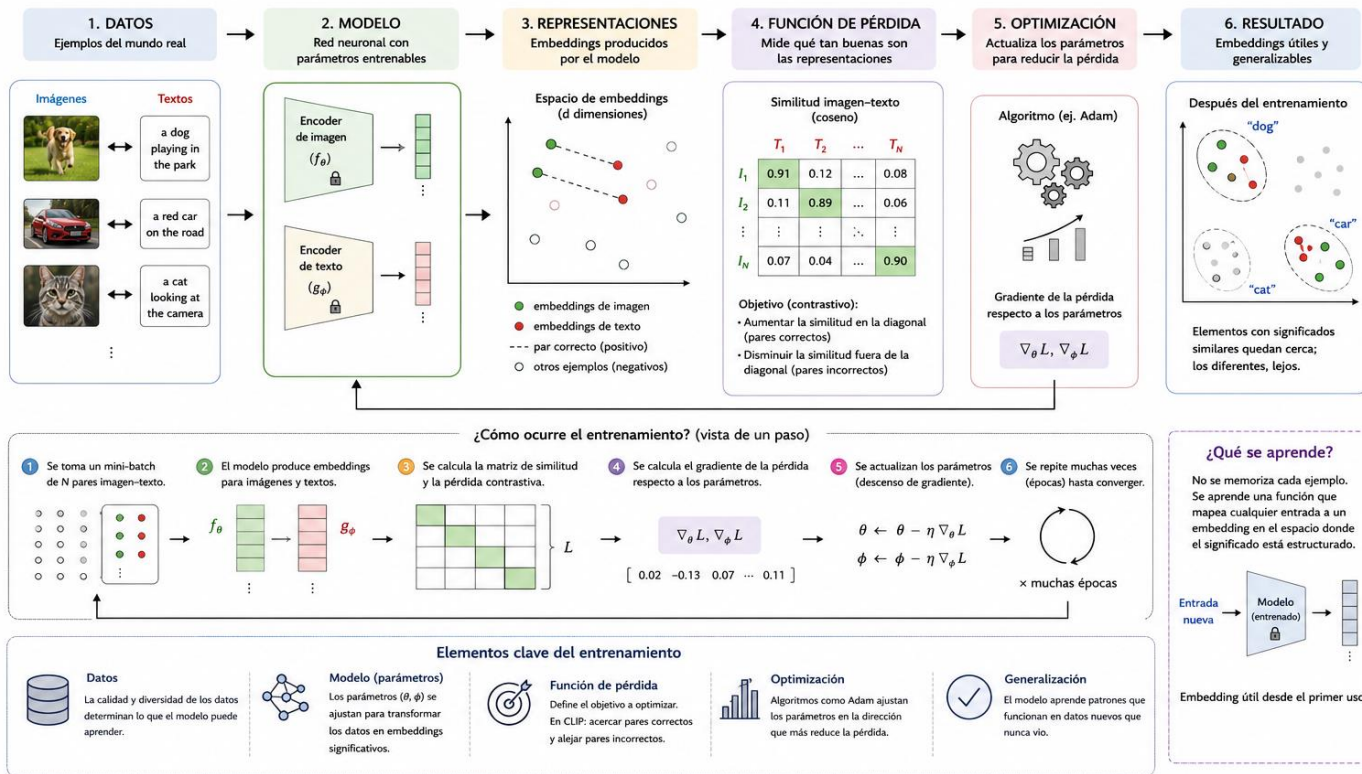
Algoritmo que calcula cuánto contribuyó cada parámetro al error y los ajusta proporcionalmente.



la loss baja con el entrenamiento



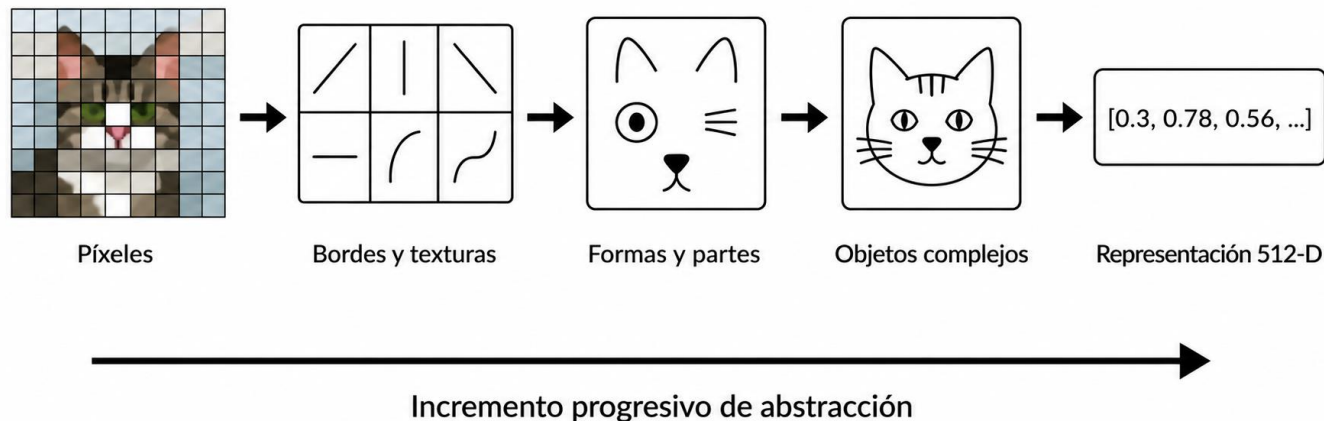
Entrenamiento



Idea clave: El entrenamiento es el proceso iterativo de minimizar una pérdida ajustando los parámetros de la red, para que las representaciones (embeddings) capturen el significado y permitan resolver tareas.

Representaciones internas

De píxeles a conceptos



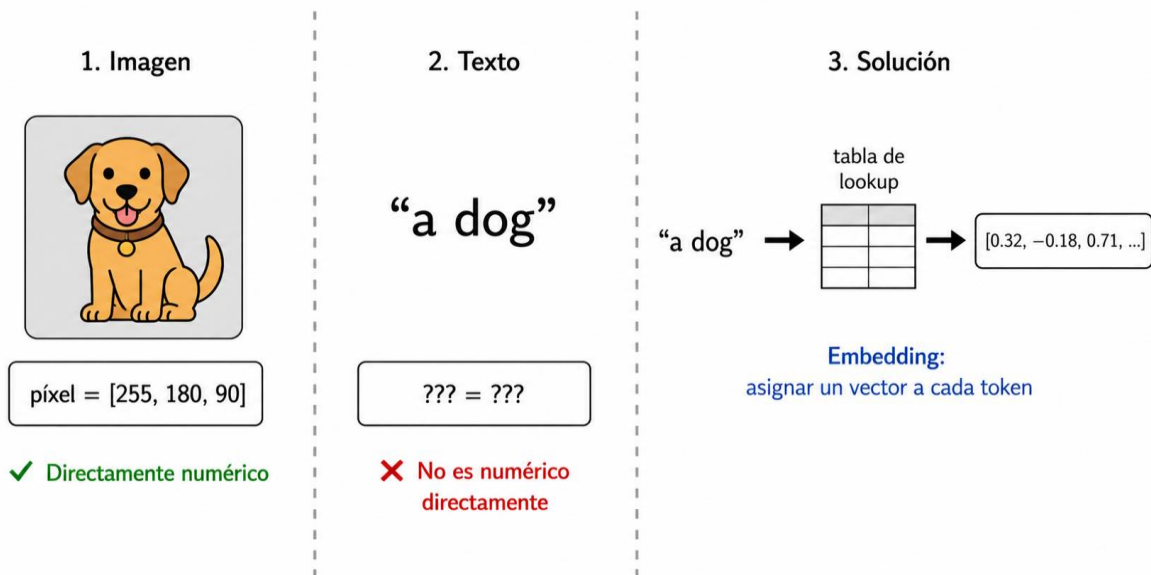
- Las primeras capas detectan bordes y colores
- Las capas intermedias combinan bordes en formas y partes
- Las capas finales representan conceptos abstractos
- El vector final es la 'comprensión' del modelo de esa imagen

El problema del texto

¿Cómo procesa una red neuronal el lenguaje?

El texto no es continuo

Las redes necesitan números, no palabras



Una imagen son números directamente:

píxel = [R, G, B] = [255, 180, 90]

Una palabra no:

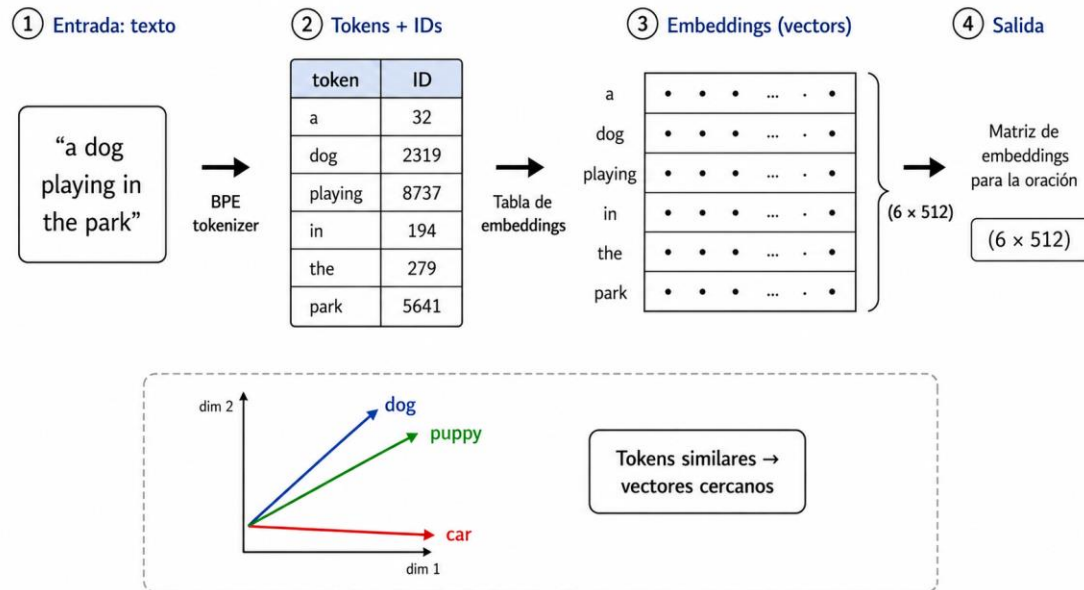
"perro" = ???

Necesitamos una función que asigne a cada palabra (o fragmento) un vector de números.

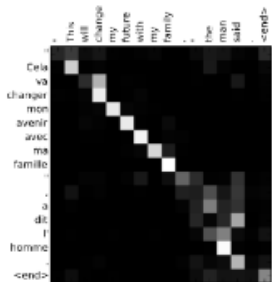
Tokenización y embeddings

Texto → tokens → vectores

- Tokenización BPE: divide el texto en sub-palabras frecuentes
- Cada token tiene un índice único en el vocabulario (49 408 tokens en CLIP)
- Cada índice apunta a un vector aprendible (embedding de dimensión 512)
- Los vectores se aprenden durante el entrenamiento — no son fijos

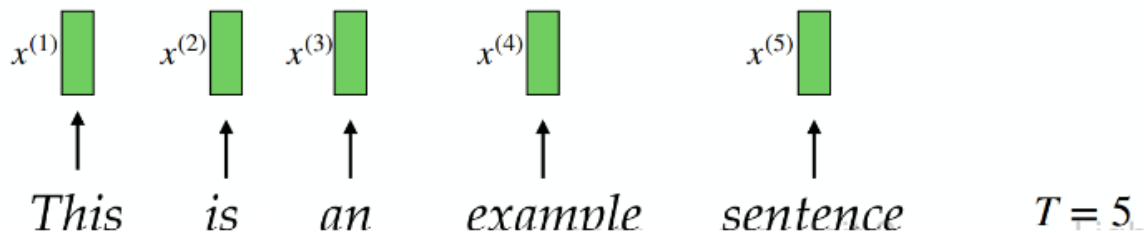


Tokenización y embeddings



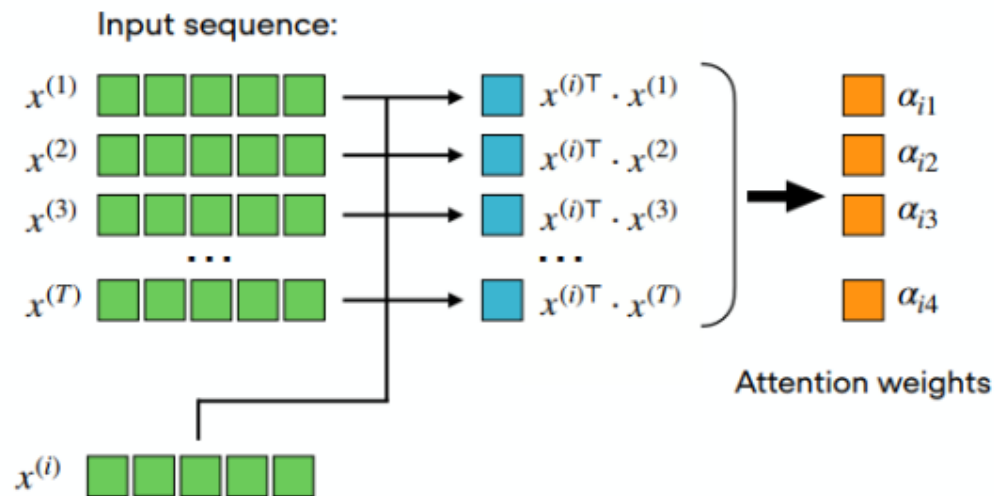
$$z^{(i)} = \sum_{j=1}^T \alpha_{ij} \cdot x^{(j)}$$

Ejemplo: $i = 2$



- Construir un vector de contexto

Attention



$$z^{(i)} = \sum_{j=1}^T \alpha_{ij} \cdot x^{(j)}$$

$$\begin{aligned}
 & x^{(1)} \text{ (green 1x5 grid) } \times \alpha_{i1} \text{ (orange 1x1 box)} \\
 + & x^{(2)} \text{ (green 1x5 grid) } \times \alpha_{i2} \text{ (orange 1x1 box)} \\
 + & x^{(3)} \text{ (green 1x5 grid) } \times \alpha_{i3} \text{ (orange 1x1 box)} \\
 & \dots \\
 + & x^{(T)} \text{ (green 1x5 grid) } \times \alpha_{i4} \text{ (orange 1x1 box)} \\
 \\
 = & \text{ (purple 1x5 grid) } z^{(i)} \\
 & \text{Context vector}
 \end{aligned}$$

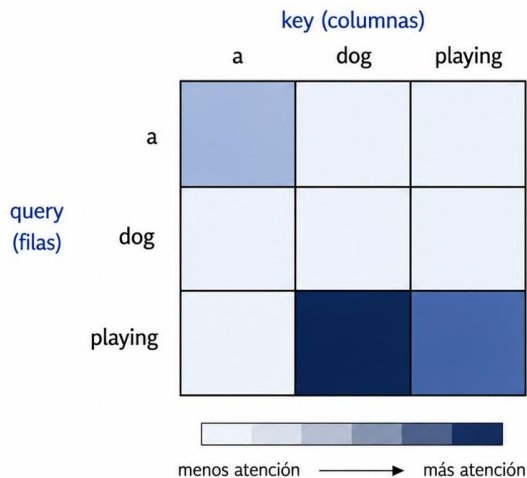
Transformer de texto

Atención: cada palabra mira a todas las demás

1. Tokens de entrada



2. Matriz de atención (3×3)
¿cuánto atiende cada token a los demás?



3. Vectores de salida
(con contexto)



contiene información
de 'dog'

Self-attention:

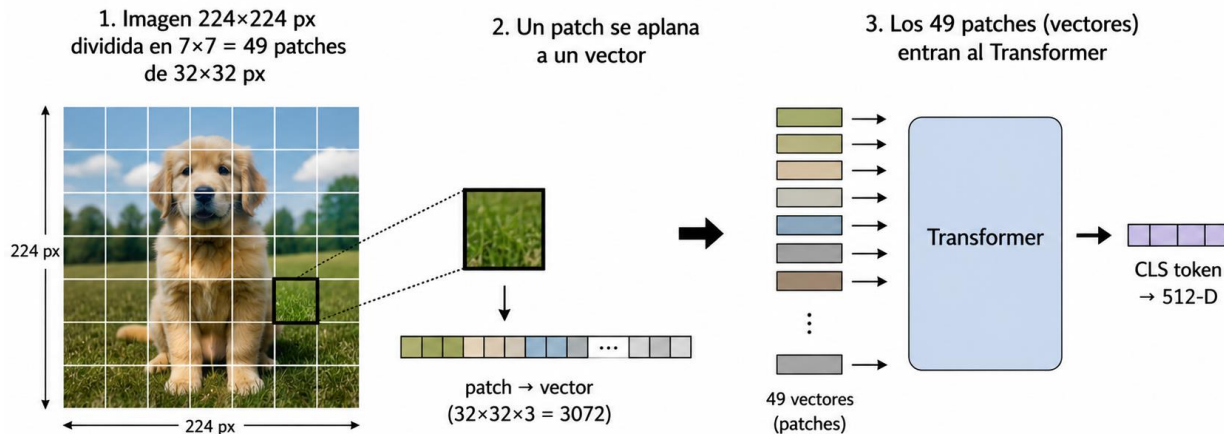
Cada token puede 'mirar' a todos los demás para construir su representación. El contexto importa — 'bank' significa algo distinto en 'river bank' vs 'bank account'.

Misma arquitectura que ViT:

patches de imagen y tokens de texto son matemáticamente equivalentes para el Transformer.

Vision Transformer (ViT)

La imagen como secuencia de patches



Idea clave:

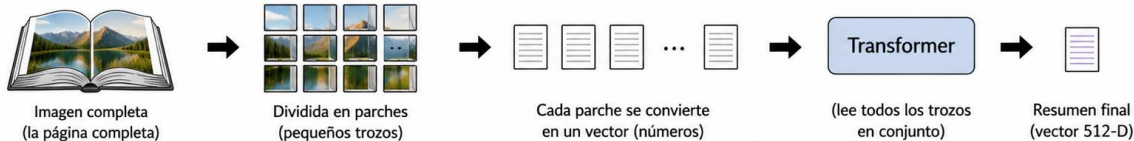
Dividir la imagen en patches de 32×32 px y tratarlos como tokens de texto.

$$224 \times 224 \text{ px} \div 32 = 49 \text{ patches}$$

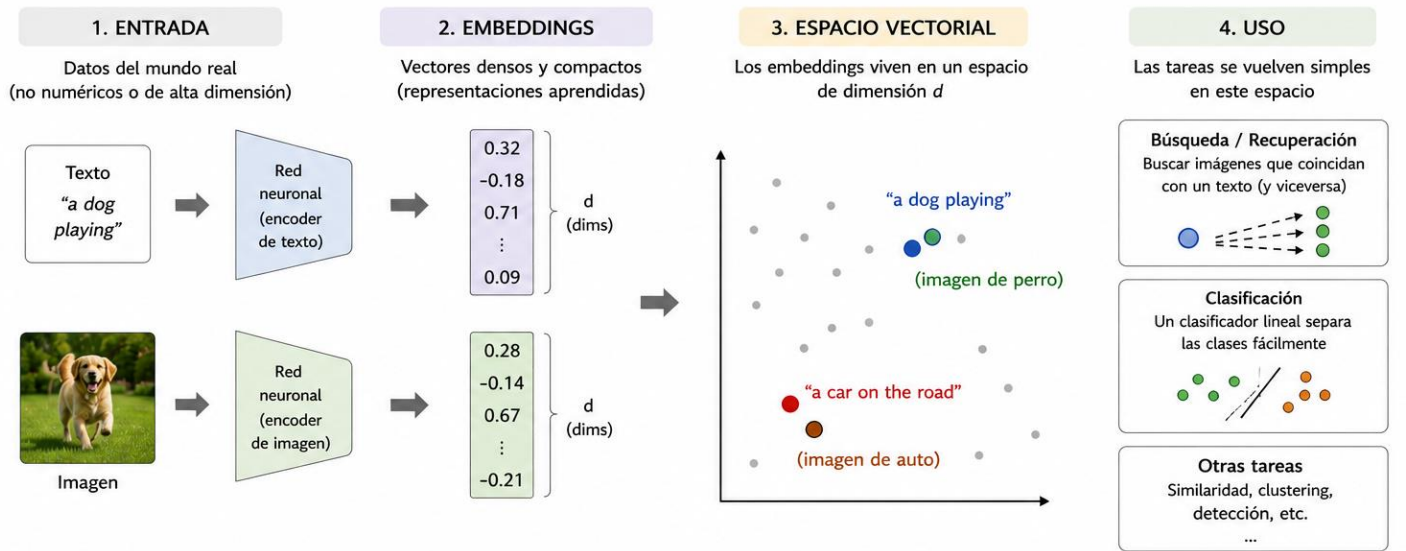
El Transformer procesa los 49 patches en paralelo, capturando relaciones entre cualquier par de regiones de la imagen — sin importar la distancia.

Mismo mecanismo que procesa texto → misma arquitectura para ambas modalidades.

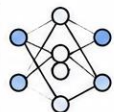
Analogía:



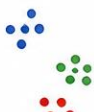
Transformers



¿Por qué funciona?



Las redes neuronales aprenden a transformar los datos de entrada en vectores que capturan su **semántica** (significado).



En este espacio, elementos con significados similares quedan **cerca**; los diferentes, **lejos**.



Esto permite resolver tareas comparando vectores con operaciones simples (similitud, producto punto, distancia, etc.).



Idea clave: Las redes neuronales no “memorizan” datos; aprenden a traerlos a un espacio donde el significado es geoméricamente estructurado.

Entrenamiento contrastivo

Maximizar diagonal, minimizar off-diagonal

1. Intuición de Contrastive Learning:

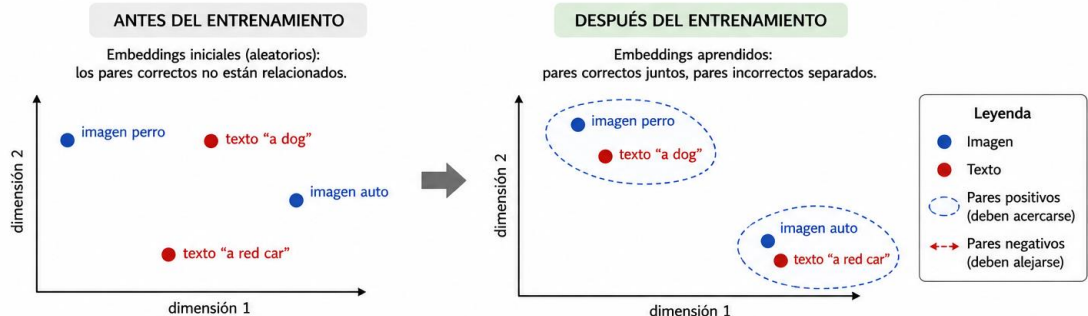
acercar pares correctos y alejar pares incorrectos en el espacio de embeddings

Un batch de N pares (imagen, texto):

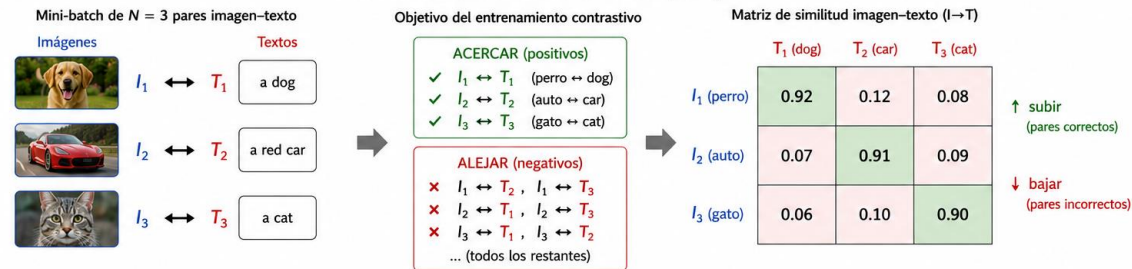
- **N pares correctos** → diagonal → sim alta
- **$N^2 - N$ pares incorrectos** → off-diagonal → sim baja

La loss optimiza simultáneamente imagen→texto y texto→imagen (cross-entropy simétrica).

Con $N=32\,768$ por batch, el modelo ve más de 1 000 millones de pares negativos por epoch.



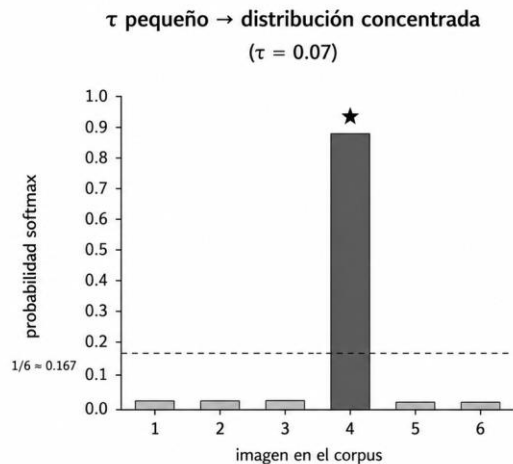
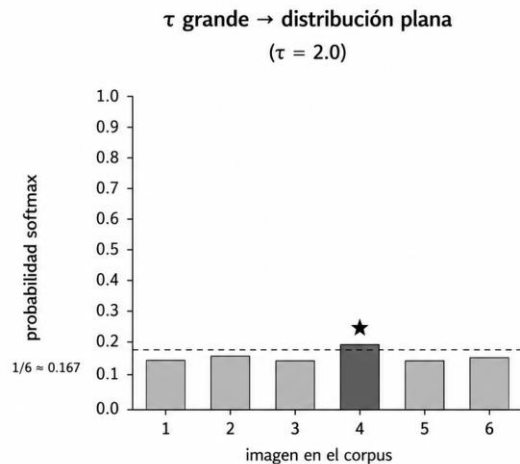
2. Cómo funciona en un mini-batch (CLIP)



Idea clave: maximizar la similitud de los pares correctos y minimizar la similitud de los pares incorrectos.

La temperatura τ

Qué tan seguros son los scores



■ barra más oscura = par correcto (imagen-texto)
★ marca el par correcto

----- probabilidad uniforme ($1/6 \approx 0.167$)

La loss escala los scores por τ antes del softmax:

$$\text{softmax}(\text{sim}(I, T) / \tau)$$

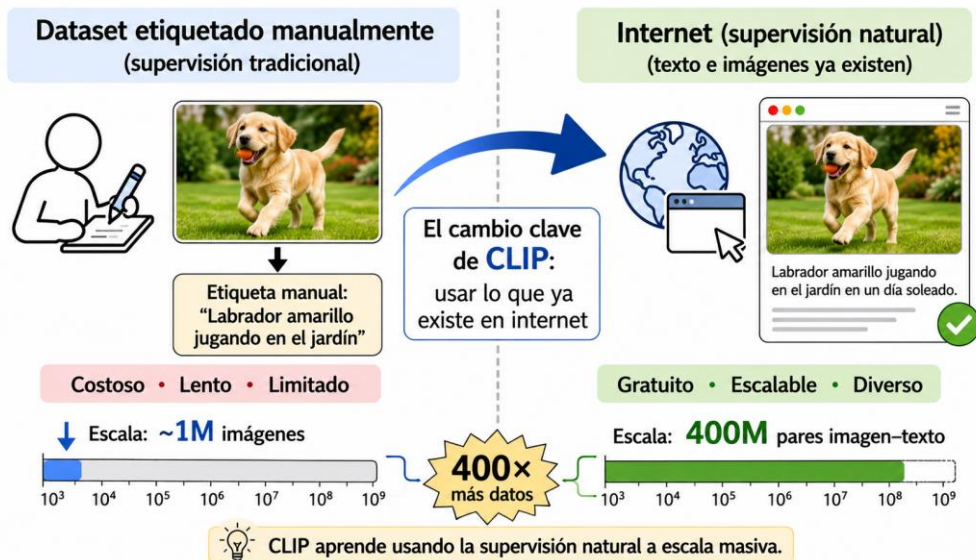
τ grande $\rightarrow$ scores más planos $\rightarrow$ aprendizaje más lento pero estable

τ pequeño $\rightarrow$ scores más contrastados $\rightarrow$ el modelo aprende diferencias finas

CLIP aprende τ como parámetro durante el entrenamiento — no se fija a mano.

Por qué funciona a escala

Supervisión natural del lenguaje



- Los datasets etiquetados manualmente son costosos y limitados en escala
- Internet tiene miles de millones de pares (imagen, texto) naturales
- El texto que acompaña a una imagen es supervisión implícita gratuita
- CLIP escala donde los datasets clásicos no pueden — 400M pares en entrenamiento
- Esto explica la capacidad zero-shot: aprendió el lenguaje visual de toda la web

Píxeles vs Representación

De datos crudos a significado

Imagen original (miniatura)



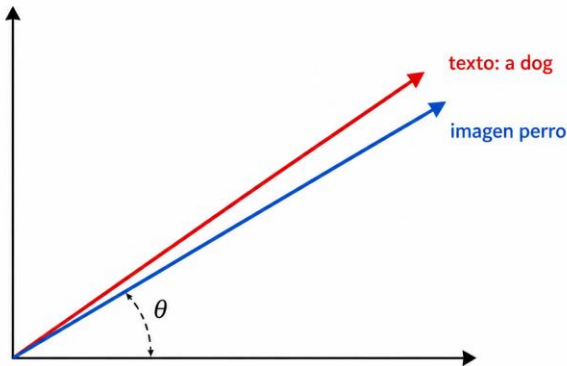
Zoom

Fragmento ampliado (cuadrícula de píxeles con valores RGB)

[255, 215, 150]	[255, 206, 130]	[255, 198, 120]	[250, 185, 105]	[280, 185, 95]	[245, 170, 90]	[235, 150, 70]	[220, 130, 60]	[200, 110, 50]	[180, 90, 40]	[160, 75, 35]
[255, 220, 160]	[255, 212, 140]	[255, 203, 125]	[255, 189, 110]	[248, 174, 95]	[238, 153, 72]	[225, 133, 62]	[225, 113, 52]	[205, 93, 42]	[185, 78, 36]	[165, 62, 30]
[255, 225, 170]	[255, 218, 150]	[255, 213, 135]	[255, 194, 115]	[250, 179, 100]	[240, 158, 75]	[230, 138, 65]	[210, 118, 55]	[190, 98, 45]	[170, 82, 38]	[150, 65, 30]
[255, 228, 175]	[255, 221, 155]	[255, 213, 140]	[255, 200, 120]	[253, 185, 105]	[243, 165, 80]	[233, 145, 70]	[215, 125, 60]	[195, 105, 48]	[175, 88, 40]	[155, 70, 32]
[255, 230, 180]	[255, 224, 160]	[255, 218, 145]	[255, 206, 128]	[255, 192, 112]	[245, 172, 86]	[235, 152, 74]	[220, 132, 62]	[200, 112, 52]	[180, 95, 42]	[160, 78, 35]
[250, 210, 160]	[245, 200, 150]	[240, 190, 140]	[235, 180, 130]	[220, 165, 110]	[195, 140, 85]	[150, 110, 65]	[100, 75, 45]	[70, 50, 30]	[60, 45, 28]	[50, 40, 25]
[220, 180, 130]	[210, 165, 120]	[200, 150, 110]	[180, 130, 90]	[140, 100, 70]	[80, 60, 45]	[35, 30, 25]	[20, 18, 15]	[10, 8, 5]	[5, 4, 3]	[3, 2, 1]
[200, 160, 115]	[190, 145, 105]	[170, 125, 85]	[130, 90, 60]	[70, 50, 35]	[25, 20, 18]	[15, 10, 14]	[10, 8, 14]	[8, 5, 16]	[5, 4, 22]	[3, 2, 35]
[190, 140, 100]	[170, 120, 85]	[140, 90, 60]	[80, 50, 35]	[40, 25, 20]	[30, 15, 16]	[20, 10, 16]	[10, 5, 25]	[8, 5, 50]	[5, 4, 70]	[3, 2, 95]
[180, 130, 95]	[150, 100, 70]	[100, 60, 45]	[60, 30, 25]	[50, 20, 18]	[40, 20, 40]	[30, 20, 90]	[20, 10, 130]	[10, 5, 200]	[8, 5, 220]	[5, 4, 210]
[150, 170, 90]	[130, 160, 80]	[110, 150, 70]	[90, 140, 60]	[70, 130, 50]	[60, 120, 45]	[50, 110, 45]	[40, 100, 60]	[30, 90, 100]	[20, 80, 100]	[10, 70, 110]

Vector de embedding (512 dimensiones)

0.32	-0.18	0.71	0.05	-0.47	0.23	-0.12	0.88	0.31	-0.34	...	0.27	-0.15	0.61	$\mathbb{R}^{512}$
primeros elementos (ejemplos)										últimos elementos (ejemplos)				



Millones de números sin contexto

Un punto en el espacio del significado

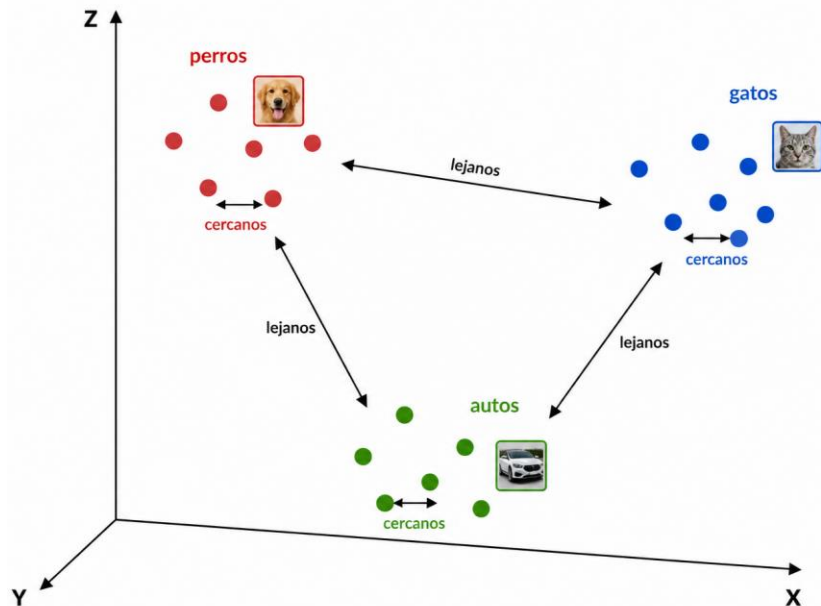
Embeddings

Representar el mundo como vectores

Un **embedding** es una función que convierte cualquier objeto

— texto, imagen, audio — en un vector de números reales en un espacio de alta dimensión.

- Objetos similares → vectores cercanos
- Objetos diferentes → vectores lejanos
- Las operaciones aritméticas tienen sentido semántico
- CLIP usa 512 dimensiones para imagen y texto



Similitud coseno

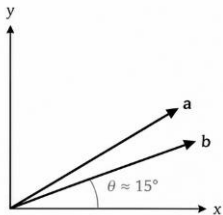
La distancia que importa

$$\text{sim}(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|) \in [-1, 1]$$

≈ 1

Similar

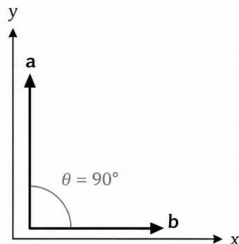
Mismo ángulo
mismo significado



≈ 0

Sin relación

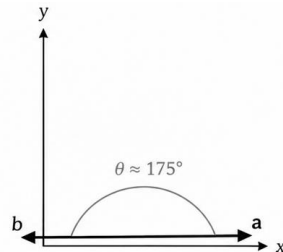
Ángulo de 90°
significados distintos



≈ -1

Opuesto

Ángulo de 180°
significados contrarios



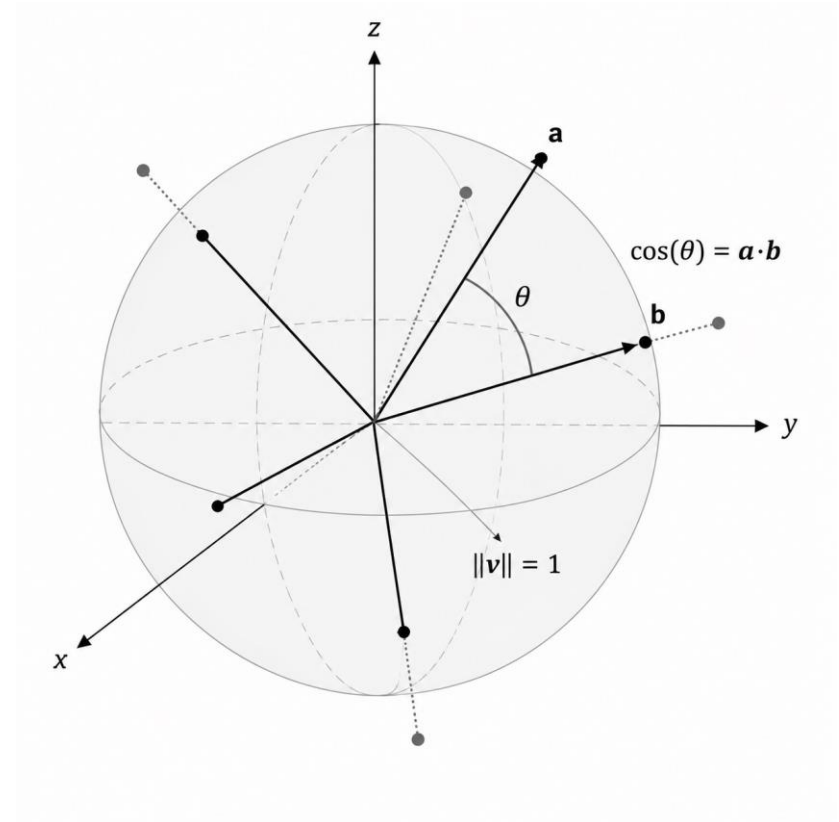
Normalización L2

Por qué el producto punto es suficiente

Si normalizamos todos los vectores ($\|v\| = 1$),
entonces:

$$\text{sim}(a, b) = a \cdot b$$

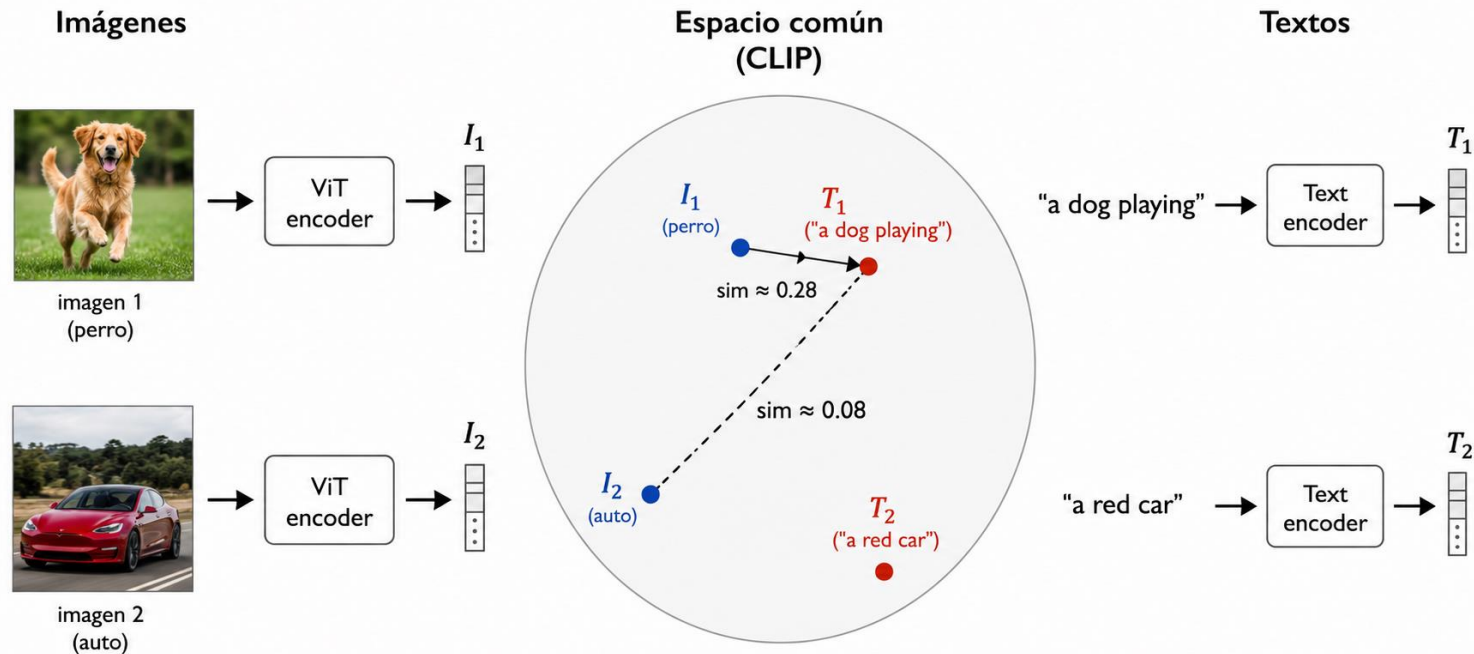
Una multiplicación matricial recupera todos los
scores del corpus.



Buscar en un corpus de 2 000 imágenes toma menos de 1 ms.

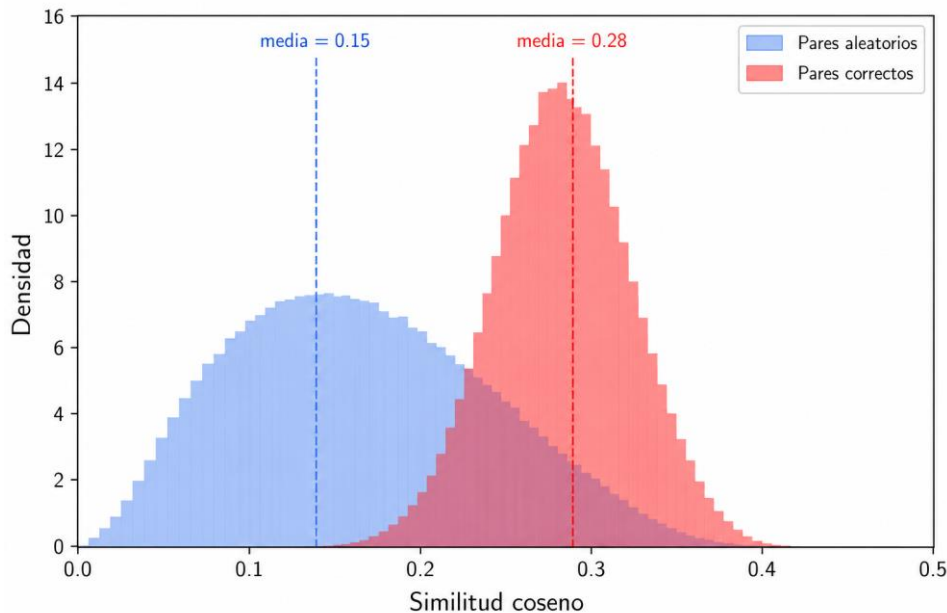
El espacio compartido de CLIP

Texto e imagen en la misma geometría



¿Funciona el espacio compartido?

Pares correctos vs pares aleatorios



Media correctos

~0.28

Media aleatorios

~0.15

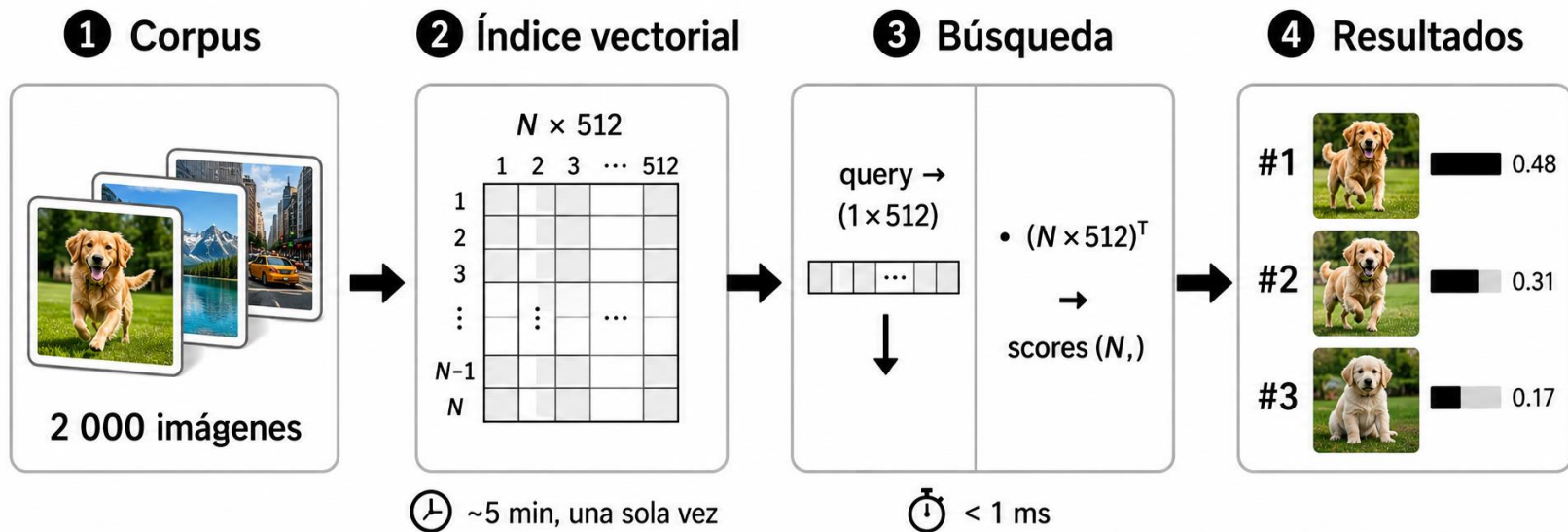
Gap

+0.13

El gap positivo confirma que la señal existe — la búsqueda semántica es posible.

El pipeline de búsqueda

Indexar $\rightarrow$ query $\rightarrow$ scores $\rightarrow$ top-k



Costo de indexación: se paga una vez. **Costo de búsqueda:** milisegundos, sin importar el corpus.

Un solo pipeline, dos modalidades

El espacio compartido hace el trabajo

Texto → Imagen

```
"a cyclist jumping" → get_text_embeddings() → compute_scores() → top-k
```

Imagen → Imagen

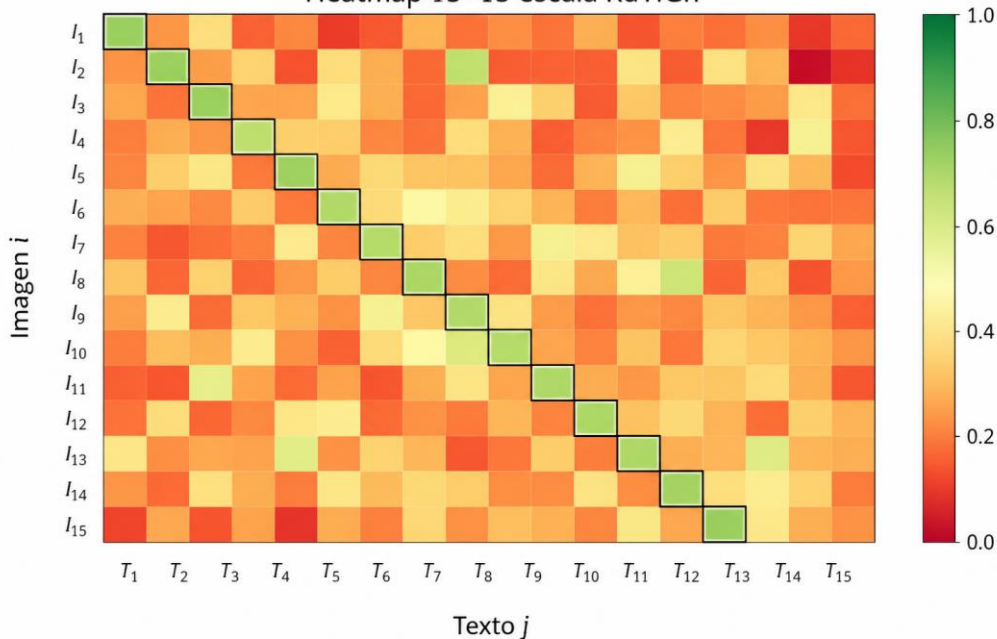
```
[imagen query] → get_image_embeddings() → compute_scores() → top-k
```

Mismo código. Distinto encoder. Esto es lo que hace especial el espacio compartido.

La matriz de similitud

$$S = I \cdot T^T \in \mathbb{R}^{N \times N}$$

Heatmap 15×15 escala RdYlGn



- $S[i,j]$ = similitud entre imagen i y texto j
- Diagonal $S[i,i]$ = par correcto
- Fuera diagonal = pares aleatorios
- Diagonal brillante confirma alineación
- Esto es exactamente lo que optimiza CLIP durante entrenamiento

Cuándo falla CLIP

Límites sistemáticos del modelo

Negaciones

"a street with no cars"

CLIP no modela ausencias — entiende "cars", ignora "no"

Conteo exacto

"exactly three people"

El modelo no cuenta, percibe presencia temática

Relaciones espaciales

"a dog to the left of a cat"

No razona sobre posiciones relativas entre objetos

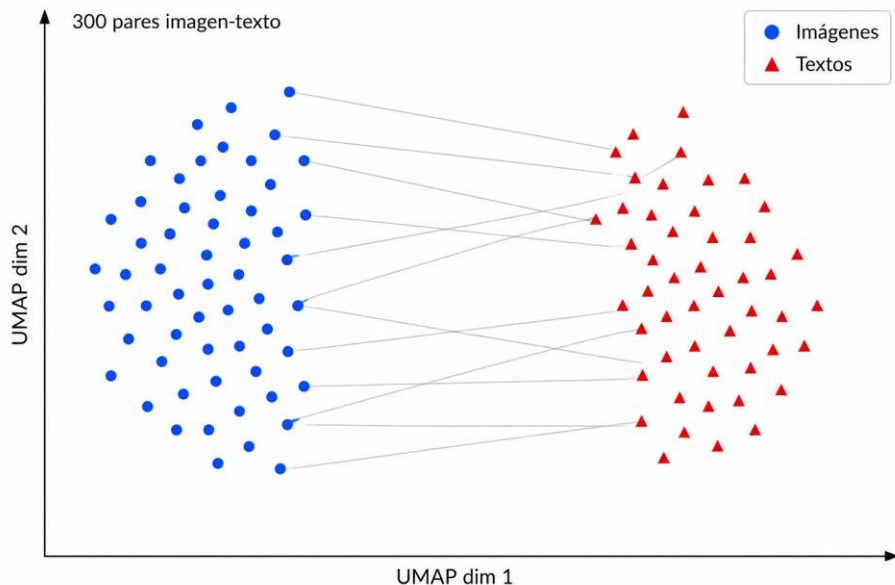
Atributos compuestos

"a red ball next to a blue box"

Mezcla atributos sin anclarlos a objetos específicos

El modality gap

Por qué texto e imagen no se mezclan en 2D



¿Qué es?

Los embeddings de imagen y texto ocupan regiones distintas de la hiperesfera, aunque sean semánticamente similares.

¿Por qué no es un bug?

El histograma lo confirma: el gap entre pares correctos y aleatorios existe en las 512 dimensiones reales. La proyección 2D engaña.

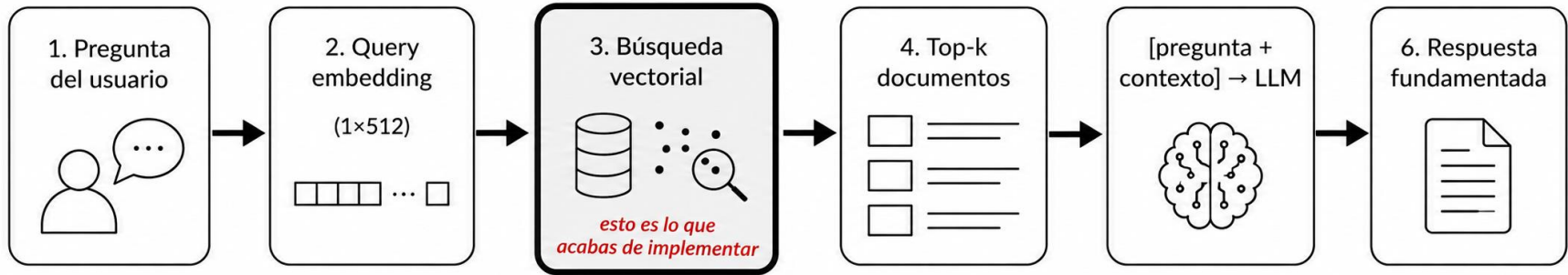
Referencia: *Liang et al., 2022 — Mind the Gap*

De retrieval a Frontier AI

El mismo mecanismo, a mayor escala

RAG — Retrieval Augmented Generation

Los LLMs no lo saben todo; el retrieval llena el gap



- El modelo de lenguaje no 'recuerda' — recupera en tiempo de inferencia
- La base de conocimiento se actualiza sin reentrenar el modelo
- ChatGPT con búsqueda, Perplexity, Copilot — todos usan RAG
- La calidad del retrieval determina la calidad de la respuesta

RAG Multimodal

Cuando la memoria incluye imágenes

RAG textual

- Corpus: documentos de texto
- Query: texto
- Embedding: solo lenguaje
- Ejemplo: Q&A sobre PDFs

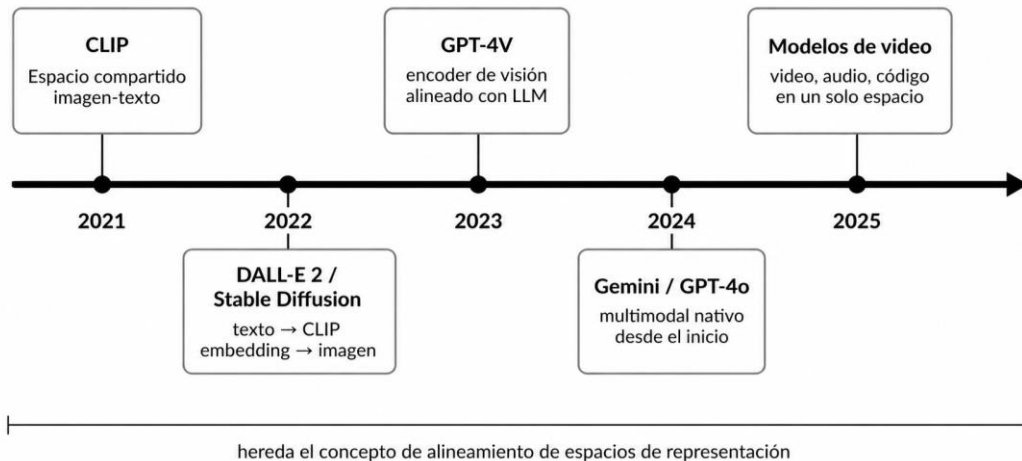


RAG multimodal

- Corpus: imágenes + texto + video
- Query: texto, imagen o ambos
- Embedding: espacio compartido (CLIP)
- Ejemplo: Google Lens, GPT-4o

Los modelos frontier usan esto

La cadena de ideas desde CLIP



- DALL-E 2 invierte el proceso: texto → embedding CLIP → imagen
- GPT-4o tiene encoder de visión alineado con el LLM — exactamente como CLIP
- Gemini y Claude nacieron multimodales: el espacio compartido es nativo
- Todos comparten la misma idea: alinear espacios de representación

Alineamiento de espacios de representación

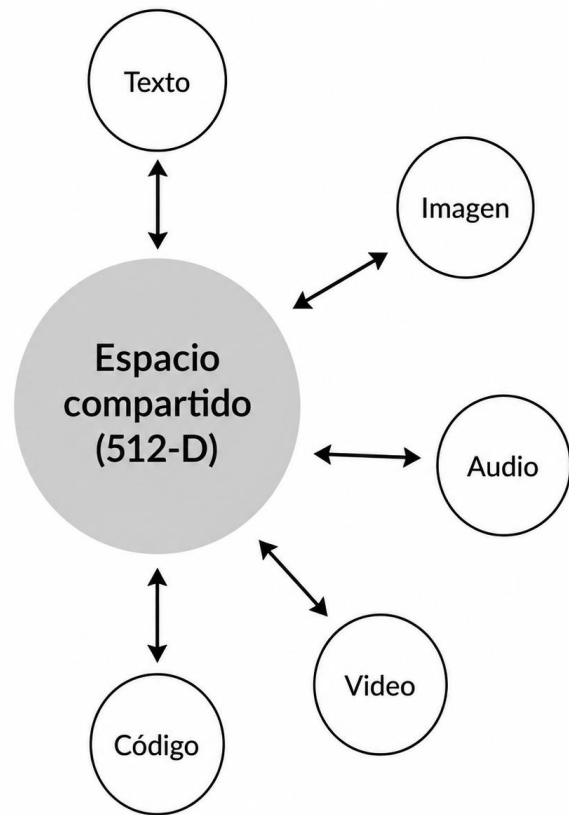
La idea unificadora de la IA moderna

La pregunta central de la IA moderna no es
'¿cómo clasifico esto?' sino

**'¿cómo represento esto de forma que sea
útil para cualquier tarea?'**

CLIP respondió esa pregunta para imagen y texto en
2021.

Los modelos frontier la están respondiendo para todas
las modalidades.



Búsqueda híbrida

Combinar texto e imagen como señal

$$\text{score}(i) = \alpha \cdot \text{sim_texto}(i) + (1-\alpha) \cdot \text{sim_imagen}(i)$$

$$\alpha = 1.0$$

Texto puro — baseline

$$\alpha = 0.0$$

Imagen pura — mismo resultado para todos

$$\alpha \in (0,1)$$

Híbrido — la imagen actúa como prior temático

Esto es lo que construirán en el notebook 03 — y lo que evaluará el leaderboard.

Ahora lo construyen ustedes

01_clip_baseline → 02_image_retrieval → 03_competition

Corpus: 2000 imágenes de Flickr30k

Métrica: Overlap@10 contra oracle de captions reales

Lo que diferencia a los equipos: la calidad del query que inventan